SC2002 Turn-based Combat Assignment Design Justification Requirement

1. Project Overview

This project is a Java-based combat simulator designed using the Entity-Control-Boundary (ECB) architectural pattern. The system manages strategic battles between a user-controlled player and waves of enemies (Goblins and Wolves) across three difficulty levels (Easy, Medium, Hard). Our primary goal was to create a system that is easy to extend (adding new characters or moves) and simple to maintain (keeping the game logic separate from the UI). The final product successfully implements all required features, including specialised actions, status effects, items, backup spawning, and cooldown mechanics.

2. System Architecture

To prevent different parts of the system from tight coupling, we organised the application into distinct packages. This ensures that the core game logic can function independently of the console interface.

2.1 Package Decomposition

The following table breaks down the system's package structure, organised by their architectural roles:

Package	Layer	Responsibility
`sc2002.game.app`	Boundary (Application)	Entry point and high-level flow controllers (`ConsoleSetupFlow`, `Main`)
`sc2002.game.domain.combat`	Entity (Combat)	Combatant hierarchy (Warrior, Wizard, Goblin, Wolf)
`sc2002.game.domain.actions`	Entity (Action)	Encapsulates all combat moves as commands that characters can execute

		during their turn
`sc2002.game.domain.status`	Entity (State)	Temporary state changes (Stun, Defend Buff, Smoke Bomb) that affect combatants
`sc2002.game.domain.items`	Entity (Resource)	Consumable resources players can use (Potion, Power Stone, Smoke Bomb) to gain advantage
`sc2002.game.domain.stats`	Entity (Data)	Specialised data layer for fixed attributes (HP, Speed, Attack) shared by all characters
`sc2002.game.engine`	Control	Coordinates game rules, turn ordering, damage calculation
`sc2002.game.level`	Control (Configuration)	Defines blueprint for each level, including which enemies spawn and when backup waves arrive
`sc2002.game.ui`	Boundary (UI)	Manages terminal interactions, including rendering menus, ASCII art, and reading user input

2.2 Key Class Responsibilities

The following table summarises the primary responsibilities of the most important classes in the
system, organised by their architectural role:

Class	Responsibility	Classification

Core Abstractions

`Combatant`	Defines the common contract for all battle participants, including health management, stat access, and damage application	Interface
`CombatAction`	Encapsulates executable actions with validation logic and uniform execution interface	Command Pattern
`StatusEffect`	Models persistent buffs or debuffs with lifecycle hooks for turn starts, turn end, and damage modification	Interface
`Item`	Represents consumable items that can be used by the player during battle	Interface

Base Classes

`AbstractCombatant`	Provides base implementation of the Combatant interface, including HP tracking, bonus stat management, and status effect handling	Base Class
`TimedStatusEffect`	Provides base implementation for status effects that expire after a set number of turns	Template Method

Controllers

`BattleEngine`	Orchestrates the complete game loop including round management, turn order	Controller

	determination, action execution, win/loss detection	
`BattleState`	Centralises game state including active enemies	State Manager

Boundary

`ConsoleBattleUI`	Handles all terminal input and output including battle display, menu rendering, and user input collection	Boundary
`ConsoleSetupFlow`	Manages the pre-game setup flow including character selection, difficulty selection, and item selection	Application Flow

Factory

`LevelFactory`	Provides static factory methods for creating Easy, Medium, and Hard level configurations	Factory Pattern

3. UML Deliverables

3.1 UML Class Diagram

The UML class diagram (**Figure A**) presents the system's architecture according to the Entity-Control-Boundary (ECB) pattern. The diagram is structured into three distinct horizontal layers: the Boundary/UI layer at the top, the Control/Engine layer in the middle, and the Domain/Entity layer at the bottom. Within the Domain layer, vertical partitioning separates Actions, Status Effects, and Items into their own columns. The project is split into 5

main packages: `application`, `domain`, `engine`, `level`, and `ui`, with `domain` containing `action`, `combat`, `item`, and `status` subpackages.

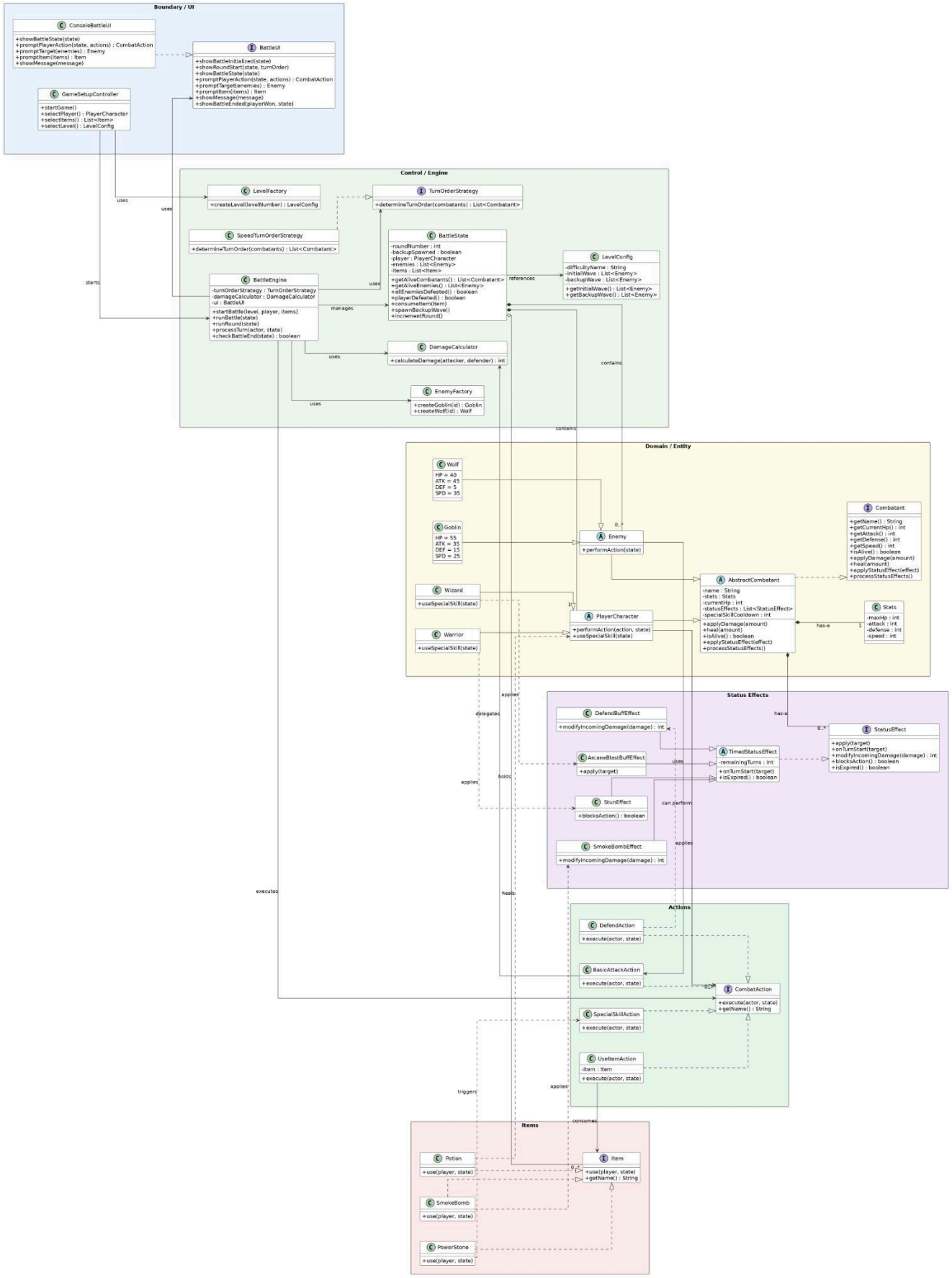

Figure A

3.2 UML Sequence Diagram

The UML sequence diagram (**Figure B**) models the complete interaction flow for the specified eight-step scenario involving a Warrior fighting a single Goblin. The flow begins with initialising both the Warrior and the Goblin. On the first turn, the Warrior uses Defend, which applies a `DefendBuff` status effect. The Goblin then uses Basic attack. The diagram shows the subsequent logic: checking whether the Goblin is alive, calculating damage after applying `DefendBuff`, and applying damage to the Warrior.

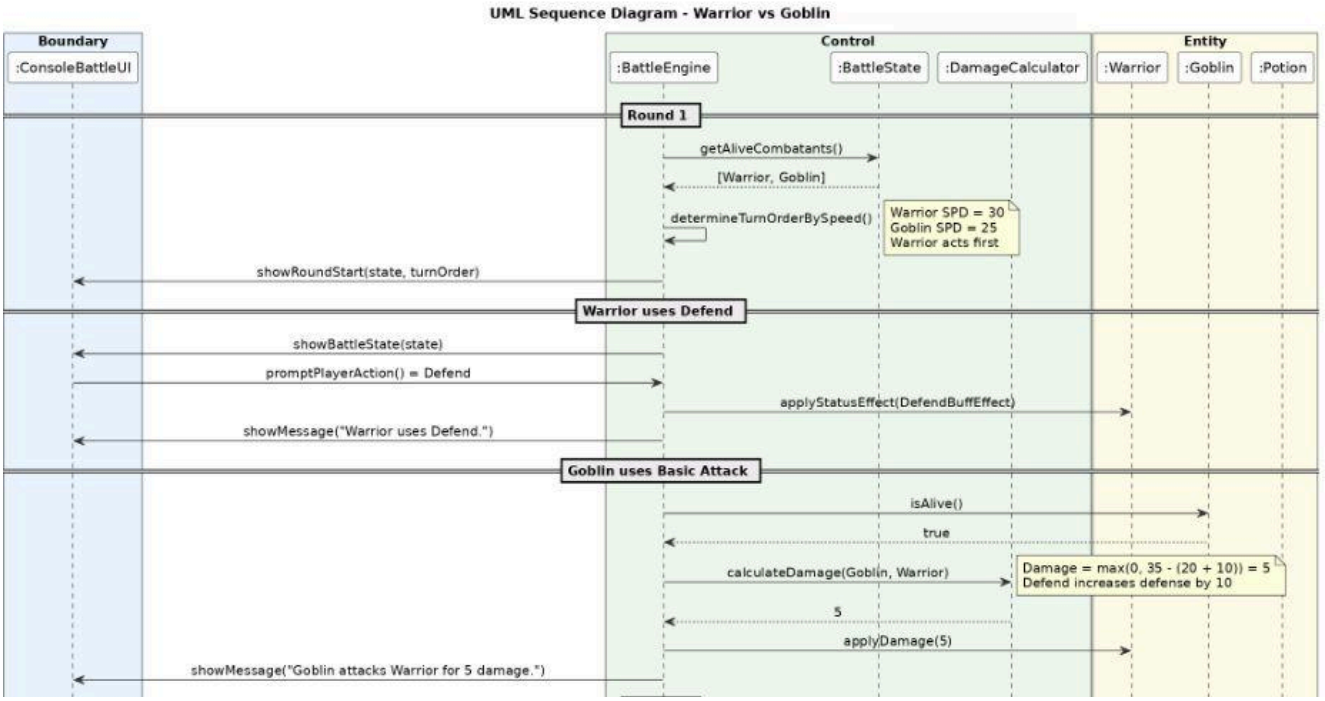

Figure B

4. Additional Features Implemented

Beyond basic requirements, the system includes several enhancements that improve user experience, robustness, and developer testing capabilities.

4.1 Enhanced Visual Presentation

The visual presentation has been significantly enhanced with ASCII art for all combatants, including the Warrior, Wizard, Goblin, and Wolf. Formatted menus with centered text, box borders, and visual separators create a polished appearance.

4.2 Improved User Experience

The user experience has been improved through intuitive navigation using W, S, A, D keys or arrow keys. The end screen provides three options for replaying with the same settings,

starting a new game, or exiting. Combatants are automatically labeled with alphabetical suffixes, so enemies appear as “Goblin A”, “Goblin B”, and “Wolf A”, with backup wave enemies additionally marked as “(Backup)” to improve situational awareness. A round summary screen collects all combat events and waits for the player to press Enter before processing, ensuring that no important information is missed during the battles.

4.3 Robust State Management

State management is particularly robust. Health points are clamped using `Math.max` and `Math.min` to ensure values never fall below zero or exceed maximum health. Status effect stacking is configurable via `isStackable`, allowing the Arcane Blast bonus to stack while other effects replace existing instances. Cooldowns decrease only on that character’s turn, not round end. Smoke Bomb grants damage mitigation until that character’s next turn.

4.4 Developer and Testing Support

For developers and testing, command-line flags provide shortcuts for previewing victory and defeat scenes without playing through entire battles. The `-no-ansi-clear` flag allows the program to run on terminals that do not support ANSI escape codes. The Wizard’s Arcane Blast skill includes defensive copying to prevent `ConcurrentModificationException` when multiple enemies are defeated in a single blast, and the attack bonus correctly persists for the entire level with proper stacking of multiple bonuses.

4.5 Memory Optimisation

Memory optimisation was achieved through the use of static final `Stats` objects for enemy types, ensuring that base statistics are loaded into memory only once per enemy type rather than once per energy instance. The use of `EnumMap` for enemy distribution in spawn waves provides better memory efficiency than standard `HashMap`.

5. SOLID Design Compliance

This section demonstrates how the system adheres to each of the five SOLID principles with specific code evidence.

5.1 Single Responsibility Principle (SRP)

Each class in the system has exactly one reason to change.

The `DamageCalculator` class, shown below (**Figure C**), is responsible only for calculating damage values and contains no battle logic or user interface code.

```java
public final class DamageCalculator {
    public int calculate(Combatant attacker, Combatant defender) {
        int raw = Math.max(0, attacker.effectiveAttack() - defender.effectiveDefense());
        return defender.statuses().modifyIncomingDamage(defender, attacker, raw);
    }
}
```

Figure C

5.2 Open-Closed Principle (OCP)

The system is open for extension but closed for modification.

New actions can be added by implementing the `CombatAction` interface without any changes to the `BattleEngine` class. The following code (**Figure D**) shows the DefendAction class as an example.

```java
public final class DefendAction implements CombatAction {
    @Override
    public ActionType type() {
        return ActionType.DEFEND;
    }

    @Override
    public String label(ActionContext context) {
        return "Defend";
    }

    @Override
    public boolean canExecute(ActionContext context) {
        return context.actor().isAlive();
    }

    @Override
    public boolean execute(ActionContext context) {
        context.actor().statuses().add(new DefendBuffEffect(), context.actor());
        context.ui().showInfo(context.actor().name() + " uses Defend (+10 DEF for current and next round).");
        return true;
    }
}
```

Figure D

Similarly, a new status effect can be added by extending `TimedStatusEffect` without modifying the `BattleEngine`.

The addition of `SmokeBombItem`, `PowerStoneItem`, and `ArcaneBlastAttackBonusEffect` all occurred without any changes to the core engine loop.

New enemy types require only extending the `Enemy` abstract class and defining its stats.

5.3 Liskov Substitution Principle (LSP)

The `PlayerCharacter` and `Enemy` classes are fully interchangeable as `Combatant` types.

The `BattleEngine` processes a list of `Combatant` objects and never needs to check the concrete type of any participant. The following code from the engine's turn processing method demonstrates the principle (**Figure E**).

```
        for (Combatant actor : turnOrder) {
            if (!actor.isAlive()) {
                continue;
            }

            if (actor == state.player()) {
                ui.showBattleState(state);
            }

            actor.statuses().onTurnStart(actor);
            if (actor.statuses().blocksAction()) {
                ui.showInfo(actor.name() + " is stunned and skips turn.");
                endActorTurn(actor);
                continue;
            }

            if (actor == state.player()) {
                handlePlayerTurn(state, state.player());
            } else {
                handleEnemyTurn(state, (Enemy) actor);
            }
```

Figure E

The `applyBasicAttack` method similarly accepts any two `Combatant` objects as attacker and defender, demonstrating that the system treats all combat participants uniformly regardless of whether they are players or enemies.

5.4 Interface Segregation Principle (ISP)

All interfaces in the system are focused and minimal.

The `StatusEffect` interface includes only lifecycle and effect modification methods (**Figure F**).

```java
public interface StatusEffect {
    String key();

    String displayName();

    boolean blocksAction();

    int modifyIncomingDamage(Combatant owner, Combatant attacker, int currentDamage);

    void onApply(Combatant owner);

    void onTurnStart(Combatant owner);

    void onTurnEnd(Combatant owner);

    default void onRemove(Combatant owner) {
        // no-op by default
    }

    void tick();

    boolean isExpired();

    default boolean isStackable() {
        return false;
    }
}
```

Figure F

5.5 Dependency Inversion Principle (DIP)

High-level modules depend on abstractions rather than concrete implementations.

The `BattleEngine` class depends on the `TurnOrderStrategy` interface, the `DamageCalculator` class, and the `BattleUI` interface. It never directly instantiates concrete strategies or UI classes. The constructor demonstrates this dependency injection (**Figure G**).

```java
public final class BattleEngine {
    private final TurnOrderStrategy turnOrderStrategy;
    private final DamageCalculator damageCalculator;
    private final BattleUI ui;
    private final EnemyFactory enemyFactory;
    private final List<PlayerTurnAction> playerTurnActions;

    public BattleEngine(TurnOrderStrategy turnOrderStrategy, DamageCalculator damageCalculator, BattleUI ui) {
        this.turnOrderStrategy = turnOrderStrategy;
        this.damageCalculator = damageCalculator;
        this.ui = ui;
        this.enemyFactory = new EnemyFactory();
        this.playerTurnActions = buildPlayerTurnActions();
    }
```

Figure G

The `Main` class creates the concrete dependencies and injects them into the engine, as shown below (**Figure H**).

```java
        BattleEngine engine = new BattleEngine(
                new SpeedTurnOrderStrategy(),
                new DamageCalculator(),
                new ConsoleBattleUI(useAnsiClear)
        );
```

Figure H

The `PlayerSkillBridge` interface further demonstrates DIP by decoupling character skills from the console UI implementation, allowing the same character classes to work with different user interface implementations.

6. Object-Oriented Design Principles

- Encapsulation: All fields in AbstractCombatant are marked private (**Figure I**). External classes modify state only through controlled methods like `applyDamage()` and `heal()`, which validate changes and maintain invariants (**Figure J**).

```java
public abstract class AbstractCombatant implements Combatant {
    private final String id;
    private final String name;
    private final CombatRole role;
    private final Stats baseStats;
    private final StatusTracker statuses = new StatusTracker();
    private final SkillCooldown skillCooldown = new SkillCooldown();
    private int currentHp;
    private int bonusAttack;
    private int bonusDefense;
```

Figure I

```java
    @Override
    public void applyDamage(int amount) {
        int clamped = Math.max(0, amount);
        currentHp = Math.max(0, currentHp - clamped);
    }
```

Figure J

- Inheritance: As shown in the UML class diagram (**Figure K**), `AbstractCombatant` serves as a base class extended by `PlayerCharacter` and `Enemy`, eliminating code duplication for common attributes like health, stats, and status effects.

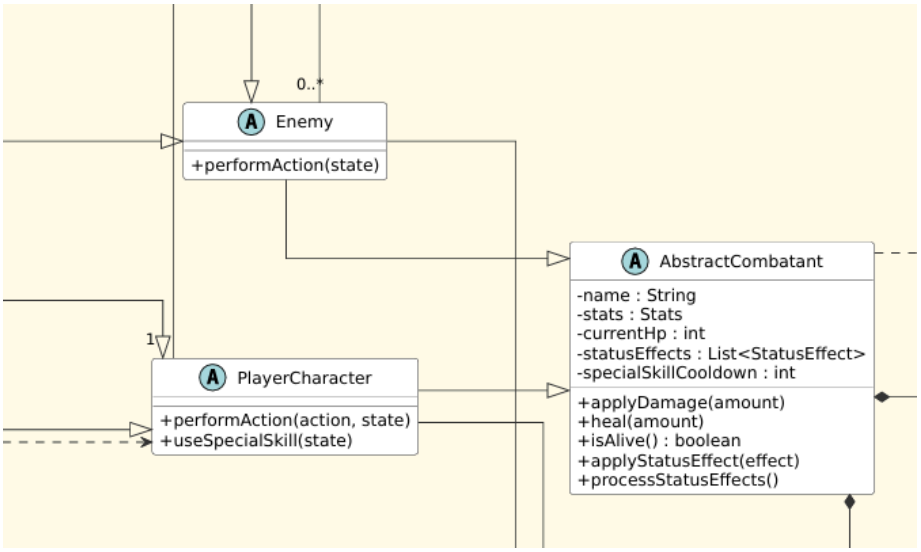

Figure K

- Polymorphism: The BattleEngine processes all combatants through the Combatant interface. The method below (**Figure L**) works identically whether the attacker is a Warrior or Wizard, and the defender is a Goblin or Wolf.

```java
    public int applyBasicAttack(Combatant attacker, Combatant defender) {
        int damage = damageCalculator.calculate(attacker, defender);
        defender.applyDamage(damage);
        return damage;
    }
```

Figure L

- Abstraction: Key behaviours are defined through interfaces (`Combatant`, `CombatAction`, `StatusEffect`, `Item`). The `BattleEngine` only depends on these interfaces rather than concrete implementations, allowing new actions, effects, or items to be added without modifying the engine.

7. Design Patterns Used

The system implements four distinct design patterns to achieve flexibility, maintainability, and extensibility.

7.1 Command Pattern (Actions)

The Command pattern is used to encapsulate all player and enemy actions via the `CombatAction` interface, allowing the `BattleEngine` to execute any action polymorphically without knowing specific details of each action type.

7.2 Template Method Pattern (Status Effects)

`TimedStatusEffect` abstract class provides the common skeleton for duration-based effects, including the tick mechanism that decrements remaining turns and the expiration check. Concrete subclasses such as `StunEffect`, `DefendBuffEffect`, and `SmokeBombInvulnerabilityEffect` override only the specific hooks they need, such as `blocksAction()` or `modifyingIncomingDamage()`. This approach eliminates code duplication across all status effects while ensuring consistent lifecycle management.

7.3 Factory Pattern (Level Creation)

`LevelFactory` class provides static methods `easy()`, `medium()`, and `hard()` that return fully configured `LevelConfig` objects. Each method encapsulates the enemy composition and backup spawn rules for that difficulty. This centralises all level configuration logic, making it easy to modify existing levels or add new ones without affecting the rest of the system.

7.4 Bridge Pattern (Player Skill Bridge)

The Bridge pattern decouples the abstraction of player skills from their implementation details. The `playerSkillBridge` interface defines methods for prompting enemy targets, applying basic attacks, and displaying information. The concrete implementation, provided by the `BattleEngine` as an inner class, connects these abstract operations to the actual UI and combat systems. This allows the `Warrior` and `Wizard` classes to remain completely

unaware of whether they are running in a console environment, a graphical interface or a test harness.

7.5 Alternative Patterns Considered

Several alternative patterns were considered during the design phase.

The State pattern was considered for managing combatant turn states but was rejected because turn state proved simple enough to handle with boolean flags and the status effect blocking mechanism.

The Visitor pattern was considered for damage calculation but would have complicated the simple arithmetic involved.

The Decorator pattern was considered for item effects but was rejected because items in this system are single-use consumables that trigger a one-time effect rather than persistent wrappers around a character. Applying a Decorator would require wrapping and then unwrapping the PlayerCharacter object, complicating inventory tracking and the item consumption lifecycle.

The Builder pattern was considered for level creation but was deemed over-engineering since only three difficulty levels were required, making the Factory pattern sufficient.

8. Design Trade-Offs

Several important trade-offs were made during the design process. The Command pattern for actions introduces multiple action classes, increasing the total class count, but this cost is justified by extensibility and adherence to the Open-Closed Principle. The `AbstractCombatant` base class creates a single inheritance hierarchy which limits flexibility compared to composition-based approaches. However, this significantly reduces code duplication across the four combatant classes while preserving Liskov Substitution. The `StatusTracker` stores effects in a simple list and iterates through them linearly for each query, which is O(n) per operation. However, the number of active effects never exceeds ten in practice, making this simpler approach preferable to more complex data structures.

The decision to limit enemies to only basic attacks was specified in the requirements, but the system remains extensible through the `CombatAction` interface for future enhancements. The console-only user interface was chosen because the assignment emphasises software

design over user interface complexity. The `BattleState` is passed as parameter to many methods, creating some coupling between components, but this centralisation of state makes testing easier and ensures state consistency throughout the combat loop.

9. Reflection

9.1 Difficulties Encountered and Solutions

Several significant challenges arose during development. The most complex issue was the semantics of status effect duration. The specification required effects to last for “the current turn and the next turn,” which created ambiguity about exactly when an effect should expire. Our solution was to have effects tick at round end and `blocksAction()` is checked before action execution. This ensures that a stun applied during a turn correctly blocks that same turn, the following round, and then expires.

The Power Stone item presented another challenge because it must trigger the player’s special skill without affecting the cooldown timer. Our solution was to create the `ItemUseSupport` interface with a `castSpecialSkill` method that accepts a `freeCast` boolean flag. The `PowerStoneItem` overrides the three-parameter version of `use` and calls the support with `freeCast` set to true, while the engine's implementation checks this flag before modifying the cooldown.

Backup spawn timing required careful coordination. The specification demands that backup waves spawn only after all initial enemies are defeated and that all backup enemies enter simultaneously. The solution implemented in `BattleState.maybeSpawnBackupWave` checks three conditions: whether a backup has already spawned, whether all current enemies are defeated, and whether the level actually has a backup wave defined. Only when all conditions are satisfied does the method spawn the complete backup wave at once.

9.2 Key Learnings from SC2002

Through this project, we realised that SOLID principles are essential tools in building a scalable software. By applying the Open-Closed Principle, we were able to integrate new features like Smoke Bomb and Power Stone items without modifying `BattleEngine`, demonstrating that the system grows through extension.

The Command pattern proved ideal for our action systems. Player moves became trivial to implement, and adding a new action now only requires creating a single class that implements the `CombatAction` interface. This separation of action definition from invocation kept the engine code clean and focused.

Similarly, the Interface Segregation Principle prevented the creation of bloated classes. The `Combatant` interface contains only the methods essential for battle, while `PlayerCharacter` adds specific functionality without forcing unnecessary requirements onto the `Enemy` class.

The Factory pattern also simplified our creation logic by centralising level configuration within the `LevelFactory`. This makes it easy to modify existing stages or pass a higher difficulty level by simply creating a new factory method.

Finally, applying the Bridge pattern to achieve dependency inversion decoupled our logic from the terminal, making the system easier to test and adapt.

9.3 Further Improvement Suggestions

Given additional development time, the following enhancements could further refine the system:

- Persistence, where we implement a save system by serialising the `BattleState` object
- Dynamic mechanics, where we implement configurable animation delays for readable battle flow and a wave counter (replacing `backupSpawned`) to support multiple backup waves for complex level designs
- Quality of life, by adding validation for healing items to prevent accidental waste of resources when health is already full

10. GitHub Repository

The complete source code for this project is available on GitHub at the following repository: https://github.com/KhiemGOM/SC2002GameFinal.git. The commit history shows individual contributions from each team member (**Figure M**).

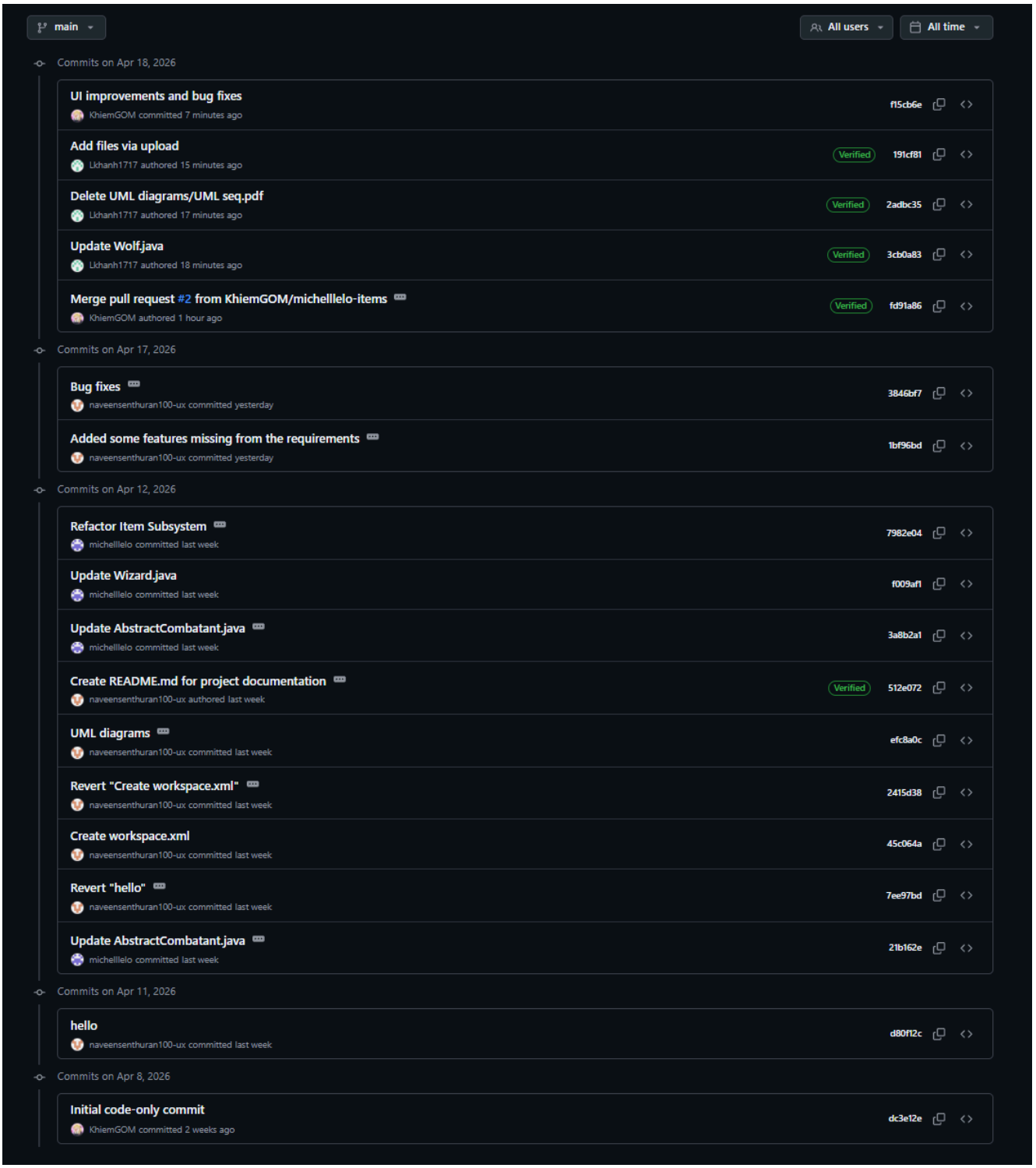

Figure M

11. Conclusion

This project demonstrates how SOLID principles and Object-Oriented Design can be used to build a fully functional turn-based game. By implementing every functional requirement (skills, status effects, and backup waves), we developed a system that is highly playable.

The integration of four key design patterns (Command, Template Method, Factory, and Bridge) ensures a flexible architecture where new actions, status, effects, items, and entire

character classes can be added with minimal changes to existing code. Our layered approach effectively decouples the UI from the game engine, making future maintenance straightforward. Ultimately, this system reflects our comprehensive understanding of the software design principles learned throughout SC2002.

Appendix A: Declaration of Original Work

Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature /Date
Nguyen Khanh	SC 2002	FAIA	
Nguyen Dang Khiem	SC 2002	FAIA	
Michelle Lo Pei Fern	SC 2002	FAIA	
Naveen Senthuran Subramanian	SC 2002	FAIA	